

DineOS — API Reference & Figma Cross-Check

Part 1: every backend endpoint, section by section, with what it's for. Part 2: what the Figma design specifies per role vs. what's actually built, backend and frontend.

Part 1 — Full API Reference

All endpoints verified directly against `dineos/urls.py` and each app's own routes — not from memory. NEW marks endpoints added in this pass.

01 — Auth · `/v1/auth/`

Method	Path	Use
POST	<code>/login/</code>	Staff login — returns JWT with role + restaurant_id claims
POST	<code>/logout/</code>	Blacklists the refresh token
POST	<code>/forgot-password/</code>	Requests a password reset token
POST	<code>/reset-password/</code>	Consumes the reset token, sets a new password
GET	<code>/me/</code>	Own profile + own tenant's 4 feature flags
POST	<code>/refresh-token/</code>	Exchanges a refresh token for a new access token
PATCH	<code>/change-password/</code>	Changes the logged-in user's own password

02 — Staff · `/v1/staff/` (Admin only)

Method	Path	Use
GET	<code>/</code>	List staff in the caller's own restaurant
POST	<code>/</code>	Create a new staff account
GET	<code>/ {id} /</code>	View one staff member
PUT/PATCH	<code>/ {id} /</code>	Edit a staff member
DELETE	<code>/ {id} /</code>	Remove a staff account
PATCH	<code>/ {id} / deactivate /</code>	Deactivate without deleting (preserves order history)

03 — Tables & Sessions · /v1/tables/

Method	Path	Use
GET	/, /{id}/	List/view tables in the caller's restaurant
GET	/qr/{slug}/{table_number}/	Customer QR landing — no auth. Resolves the table + its current session
GET	/ {id} /session/	The table's currently open session (404 once closed)
POST	/ {id} /session/	Start a session (idempotent — returns the existing one if already open)
DELETE	/ {id} /session/	Manager override — force-close without payment
POST	/ {id} /bill-request/	Customer asks for the bill
PATCH	/ {id} /status/	Manager override — force a table's status
GET	/ {id} /history/	Past closed sessions for this table

04 — Menu · /v1/menu/

Method	Path	Use
GET	/customer/{table_id}/	Customer's menu view — no auth, portions-aware
GET	/all/	Full management view — including unavailable items
GET	/categories/	List categories
POST	/categories/	Create a category
PUT	/categories/{id}/	Rename/edit a category
DELETE	/categories/{id}/	Delete (409 if it still has items)
PATCH	/ {id} /availability/	Toggle a dish on/off the menu
GET	/ {id} /	View one menu item
PUT	/ {id} /	Edit a menu item
DELETE	/ {id} /	Delete a menu item
GET	/	Order-taking menu (Server/Cashier) — same shape as the customer view
POST	/	Create a menu item (Manager)

05 — Prepared Dishes · /v1/prepared-dishes/

Method	Path	Use
PATCH	/ {dish_id} /add-portions/	Top up today's available count for a dish
GET	/today/	Today's prep log — every tracked dish's remaining count

06 — Orders · /v1/orders/

Method	Path	Use
POST	/	Place an order (customer or staff — same endpoint)
GET	/active/	Kitchen's live queue (KDS key)
GET	/ready/	Orders ready to collect, across every table
GET	/session/{session_id}/	All rounds for one table session
GET	/table/{table_id}/	All orders for a table (Server)
PATCH	/ {order_id} /status/	Kitchen advances New→Accepted→Preparing→Ready (KDS key)
PATCH	/ {order_id} /collected/	Server picked it up from the kitchen
PATCH	/ {order_id} /served/	Server delivered it to the table
GET	/ {order_id} /	View one order

07 — Billing · /v1/bills/

Method	Path	Use
GET	/session/{session_id}/	Bill preview/detail (Cashier — staff-only, customer never sees amounts)
POST	/payment/	Confirm payment, close the session (idempotent)

07b — Cashier Shift · /v1/cashier/ NEW

Built this pass, from the Figma "Cashier Home / Cash Reconciliation / Daily Collections" screens — there was no shift/session concept for cashiers before; only per-table billing existed.

Method	Path	Use
POST	/shifts/open/	Start a shift (idempotent — returns the existing open one)
GET	/shifts/current/	"Cashier Home" dashboard — collected today, awaiting/active tables, paid-today count
GET	/shifts/{id}/reconciliation/	System totals by payment method (Cash/Card/UPI) for the "App says ₹X" comparison
POST	/shifts/{id}/close/	Submit the physical cash count; 409 with the discrepancy if it doesn't match unless acknowledged
GET	/collections/daily/	"Daily Collections" report — totals, breakdown, today's bills, vs. yesterday

08 — Notifications · `/v1/notifications/`

Method	Path	Use
GET	<code>/</code>	List notifications for the logged-in user
GET	<code>/unread-count/</code>	Badge count
PATCH	<code>/{"id"}/read/</code>	Mark one as read

09 — Health

Method	Path	Use
GET	<code>/healthz/</code>	Liveness check

10 — Platform / Super Admin · `/platform/`

Method	Path	Use
POST	<code>/auth/login/</code>	Super Admin login — separate auth realm entirely
GET	<code>/tenants/</code>	List every client restaurant on the platform
POST	<code>/tenants/</code>	Onboard a new client restaurant
GET	<code>/tenants/{id}/</code>	View one tenant's full record
PATCH	<code>/tenants/{id}/</code>	Edit a tenant — most commonly, flip a feature flag
DELETE	<code>/tenants/{id}/</code>	Remove a tenant

WebSocket routes (Channels)

Path	Consumer	Auth	Use
<code>/ws/staff/</code>	StaffConsumer	JWT via <code>?token=</code>	Live order/table/notification/payment events by role
<code>/ws/kitchen/</code>	KitchenConsumer	KDS key via <code>?kds_key=</code>	Live new-order feed for the kitchen tablet
<code>/ws/table/{session_id}/</code>	TableConsumer	None (session UUID is the secret)	Live order-status updates for one customer's table

Note: an order's *placement* broadcasts only to the kitchen group, not the customer — the customer already knows they just ordered from the REST response. What reaches them live is subsequent *status changes* (accepted/preparing/ready/served), confirmed against a real running backend in this pass.

Total: 45 REST endpoints (5 new this pass) + 3 WebSocket routes. Backed by 56 passing pytest tests and a 73-request, 0-failure Postman/Newman run against a live server.

Part 2 — Figma vs. What's Built

Cross-checked against the Figma file's screen inventory (fileKey `mu9bTCiU0rBNp0SCt9DuMi`) — 90 screens across 6 role clusters. Status reflects: does a backend API exist for it, and does a Flutter screen exist for it.

Role	Figma screens (examples)	Backend API	Flutter frontend
Customer	Welcome/Table Landing, Menu Browsing, Order Tracking, Order Review, Bill and Feedback	MOSTLY — ordering/billing covered; no feedback API	BUILT — Phase 1
Server	Table Overview, Table Detail, Take Order, Orders Ready, Notifications Centre	FULL	BUILT — Phase 1
Kitchen	KOT Live Grid, Order Detail	FULL	NOT STARTED — Phase 2
Cashier	Cashier Home, Table Bill/Payment, Receipt, Bill Detail (Past Bills), Daily Collections, Cash Reconciliation	MOSTLY — shift/dashboard/reconciliation/collections built this pass; no dedicated past-bill search/receipt-number endpoint	NOT STARTED — Phase 2
Manager	Dashboard, Menu/Category Management, Item Detail, Availability, + Inventory, Billing Overview, Prep-log frames	MOSTLY — menu/category/prep-log covered; no inventory API	NOT STARTED — Phase 3
Admin	15 screens — AI Chat through Profile and Account	PARTIAL — staff CRUD exists; no AI chat, no self-service profile-edit beyond password	NOT STARTED — Phase 3

Specific gaps found

- **Customer feedback** — Figma's "Bill and Feedback" screen implies a post-meal rating/feedback submission; no backend endpoint exists. Matches the already-known Phase 2 backend scope (customer feedback).
- **Past-bill search** — Figma's "Bill Detail (Past Bills)" implies searching by bill ID/table across any date. `/v1/cashier/collections/daily/` accepts a `?date=` param so it can pull any single day, but there's no ID/table-number search across bills.
- **Inventory tracking** — several Manager-cluster frames (Inventory) have no backend equivalent — matches the already-known Phase 2 scope.
- **AI Chat (Admin)** — no backend endpoint; matches the already-known "real Groq/AI integration" Phase 2 item (currently mocked, per `LLM_PROVIDER=mock`).
- **Receipt/bill display numbers** — the Receipt and Table Bill screens show human-friendly codes ("Bill #143", "#TXN-8842"); the backend only has a UUID `Bill.id` . Cosmetic gap, not a functional one.
- **Super Admin** — no "Super Admin" section was found anywhere in this Figma file's screen inventory; the platform-management screens (tenant list/create/flag-toggles) covered by `docs/Flutter_Frontend_Plan.pdf` 's Phase 4 don't appear to have a corresponding design yet. Worth confirming whether that's designed elsewhere or still needed.

Frontend build status, plainly

Only **Customer** and **Server** have Flutter screens built (Phase 0 + Phase 1). **Kitchen, Cashier, Manager, and Admin have zero frontend screens** — this was always the planned phasing (Phases 2–3 in `Flutter_Frontend_Plan.pdf`), not a gap that appeared unexpectedly. The backend is now ready for a Cashier frontend to be built against it, once that phase starts.

DineOS — API Reference & Figma Cross-Check · generated from the live codebase and the Figma file directly